TEAM 4 – Ctrl+Alt+Deploy

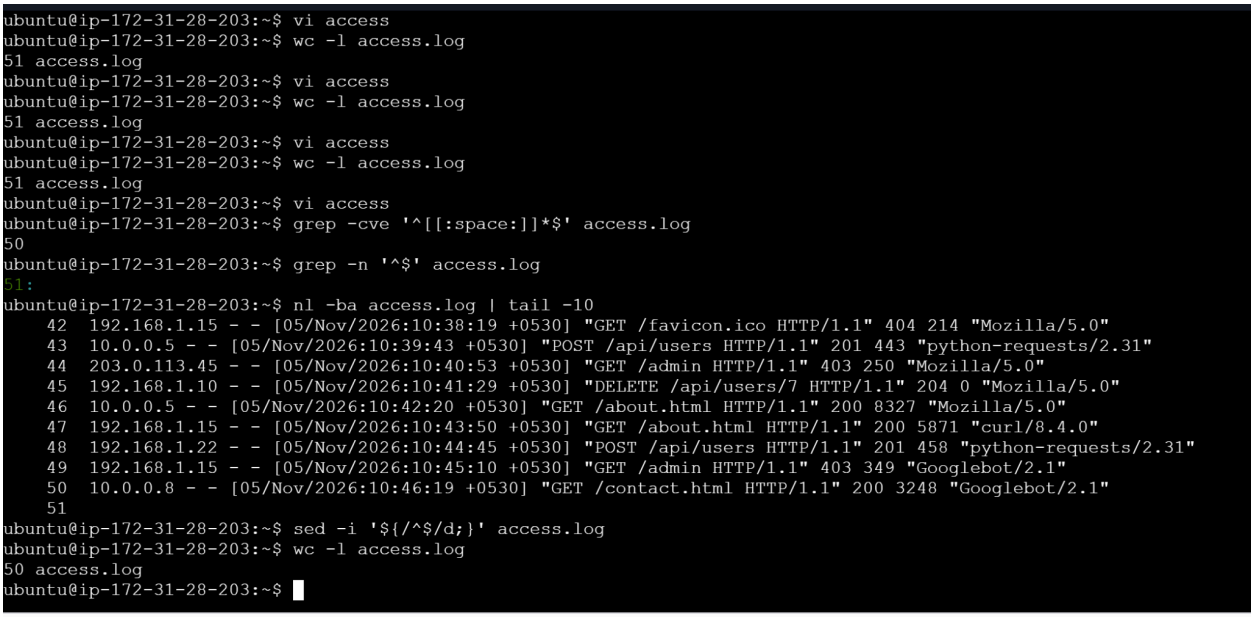
LINUX TEXT PROCESSING – SESSION 2 REPORT

grep, cut and awk Task

Session Date: 06 November 2026
Platform: AWS EC2
Operating System: Ubuntu Linux
Team Name: Ctrl-Alt-Delete
Attendance: 6 Members

*In today's class, we are going to learn Linux text processing commands such as GREP, CUT, and AWK.
We will perform hands-on exercises using an `access.log` file containing 50 log records.
We will practice searching, filtering, extracting, and analyzing log data.
We will also learn to identify errors, count requests, and generate reports using these commands.

Opened the `access.log` file using the `vi` editor and pasted all 50 sample log records. Saved the file and exited the editor successfully. Then executed the `wc -l access.log` command to verify the total number of lines. After identifying and removing one extra blank line, I successfully confirmed that the file contains exactly 50 log records.



```
ubuntu@ip-172-31-28-203:~$ vi access
ubuntu@ip-172-31-28-203:~$ wc -l access.log
51 access.log
ubuntu@ip-172-31-28-203:~$ vi access
ubuntu@ip-172-31-28-203:~$ wc -l access.log
51 access.log
ubuntu@ip-172-31-28-203:~$ vi access
ubuntu@ip-172-31-28-203:~$ wc -l access.log
51 access.log
ubuntu@ip-172-31-28-203:~$ vi access
ubuntu@ip-172-31-28-203:~$ grep -cve '^[[:space:]]*$' access.log
50
ubuntu@ip-172-31-28-203:~$ grep -n '^$' access.log
51:
ubuntu@ip-172-31-28-203:~$ nl -ba access.log | tail -10
    42  192.168.1.15 - - [05/Nov/2026:10:38:19 +0530] "GET /favicon.ico HTTP/1.1" 404 214 "Mozilla/5.0"
    43  10.0.0.5 - - [05/Nov/2026:10:39:43 +0530] "POST /api/users HTTP/1.1" 201 443 "python-requests/2.31"
    44  203.0.113.45 - - [05/Nov/2026:10:40:53 +0530] "GET /admin HTTP/1.1" 403 250 "Mozilla/5.0"
    45  192.168.1.10 - - [05/Nov/2026:10:41:29 +0530] "DELETE /api/users/7 HTTP/1.1" 204 0 "Mozilla/5.0"
    46  10.0.0.5 - - [05/Nov/2026:10:42:20 +0530] "GET /about.html HTTP/1.1" 200 8327 "Mozilla/5.0"
    47  192.168.1.15 - - [05/Nov/2026:10:43:50 +0530] "GET /about.html HTTP/1.1" 200 5871 "curl/8.4.0"
    48  192.168.1.22 - - [05/Nov/2026:10:44:45 +0530] "POST /api/users HTTP/1.1" 201 458 "python-requests/2.31"
    49  192.168.1.15 - - [05/Nov/2026:10:45:10 +0530] "GET /admin HTTP/1.1" 403 349 "Googlebot/2.1"
    50  10.0.0.8 - - [05/Nov/2026:10:46:19 +0530] "GET /contact.html HTTP/1.1" 200 3248 "Googlebot/2.1"
    51
ubuntu@ip-172-31-28-203:~$ sed -i '${/^$/d;}' access.log
ubuntu@ip-172-31-28-203:~$ wc -l access.log
50 access.log
ubuntu@ip-172-31-28-203:~$
```

What is GREP?

GREP (Global Regular Expression Print) is a Linux command used to search for specific words, patterns, or text in files and display the matching lines.

Common GREP Flags

Flag	Description
`-i`	Ignore uppercase and lowercase differences while searching.
`-c`	Counts and displays the number of matching lines.
`-v`	Displays lines that do not match the given pattern.
`-n`	Displays matching lines along with their line numbers.
`-l`	Displays only the names of files containing the matching text.
`-o`	Displays only the matched text instead of the entire line.
`-r`	Searches recursively inside files in directories and subdirectories.

CUT Command

The `CUT` command in Linux is used to extract specific fields, columns, or characters from a file. It helps retrieve required information such as IP addresses, HTTP methods, timestamps, status codes, and user agents from log files.

Common CUT Flags:

Flag	Description
`-d`	Specifies the delimiter used to separate fields.
`-f`	Selects specific fields or columns.
`-c`	Extracts specific character positions.
`-b`	Extracts specific bytes from each line.

AWK Command

The `AWK` command in Linux is used for text processing, filtering, and analyzing structured data. It helps extract multiple fields, calculate total and average response sizes, count requests based on IP addresses and status codes, and generate formatted reports using `printf`.

Common AWK Features:

Feature	Description
`$0`	Represents the entire line.
`$1, $2`	Represents the first and second fields.
`NR`	Represents the current record (line) number.
`NF`	Represents the total number of fields in a record.
`BEGIN`	Executes commands before processing input records.
`END`	Executes commands after processing all input records.
`printf`	Prints formatted output.

Section A: grep (Questions 1 to 10)

[HEMA SRI]

Q1. Show every request that returned status code 404.

Command: grep ' 404 ' access.log

Output:

```
ubuntu@ip-172-31-28-203:~$ grep ' 404 ' access.log
203.0.113.45 - - [05/Nov/2026:10:01:01 +0530] "GET /missing.html HTTP/1.1" 404 243 "python-requests/2.31"
203.0.113.45 - - [05/Nov/2026:10:13:10 +0530] "GET /favicon.ico HTTP/1.1" 404 293 "curl/8.4.0"
192.168.1.10 - - [05/Nov/2026:10:14:20 +0530] "GET /missing.html HTTP/1.1" 404 191 "python-requests/2.31"
10.0.0.8 - - [05/Nov/2026:10:18:56 +0530] "GET /favicon.ico HTTP/1.1" 404 199 "Googlebot/2.1"
172.16.0.3 - - [05/Nov/2026:10:25:57 +0530] "GET /favicon.ico HTTP/1.1" 404 284 "curl/8.4.0"
192.168.1.15 - - [05/Nov/2026:10:38:19 +0530] "GET /favicon.ico HTTP/1.1" 404 214 "Mozilla/5.0"
```

Q2. Count how many POST requests are in the log.

Command: grep -c 'POST' access.log

Output:

```
ubuntu@ip-172-31-28-203:~$ grep -c 'POST' access.log
11
```

Q3. Show all requests made from the IP address 192.168.1.15.

Command:grep '^192.168.1.15 ' access.log

Output:

```
ubuntu@ip-172-31-28-203:~$ grep '^192.168.1.15' access.log
192.168.1.15 - - [05/Nov/2026:10:04:33 +0530] "GET /images/logo.png HTTP/1.1" 200 20573 "curl/8.4.0"
192.168.1.15 - - [05/Nov/2026:10:05:51 +0530] "PUT /api/users/12 HTTP/1.1" 200 13815 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:07:08 +0530] "GET /images/logo.png HTTP/1.1" 200 21454 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:10:26 +0530] "GET /contact.html HTTP/1.1" 200 2120 "python-requests/2.31"
192.168.1.15 - - [05/Nov/2026:10:17:51 +0530] "GET /index.html HTTP/1.1" 200 13221 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:19:45 +0530] "GET /contact.html HTTP/1.1" 200 6625 "Googlebot/2.1"
192.168.1.15 - - [05/Nov/2026:10:24:52 +0530] "GET /admin HTTP/1.1" 403 266 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:30:08 +0530] "GET /Admin/settings HTTP/1.1" 403 218 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:38:19 +0530] "GET /favicon.ico HTTP/1.1" 404 214 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:43:50 +0530] "GET /about.html HTTP/1.1" 200 5871 "curl/8.4.0"
192.168.1.15 - - [05/Nov/2026:10:45:10 +0530] "GET /admin HTTP/1.1" 403 349 "Googlebot/2.1"
```

Q4. Show every line that does NOT have status code 200.

Command:grep -v ' 200 ' access.log

Output:

```
ubuntu@ip-172-31-28-203:~$ grep -v '200' access.log
203.0.113.45 - - [05/Nov/2026:10:01:01 +0530] "GET /missing.html HTTP/1.1" 404 243 "python-requests/2.31"
192.168.1.10 - - [05/Nov/2026:10:01:48 +0530] "GET /api/orders HTTP/1.1" 500 330 "Mozilla/5.0"
10.0.0.8 - - [05/Nov/2026:10:07:37 +0530] "GET /Admin/settings HTTP/1.1" 403 327 "Mozilla/5.0"
192.168.1.10 - - [05/Nov/2026:10:09:35 +0530] "POST /api/orders HTTP/1.1" 503 447 "Mozilla/5.0"
10.0.0.5 - - [05/Nov/2026:10:11:21 +0530] "GET /old-page HTTP/1.1" 301 209 "Googlebot/2.1"
203.0.113.45 - - [05/Nov/2026:10:13:10 +0530] "GET /favicon.ico HTTP/1.1" 404 293 "curl/8.4.0"
192.168.1.10 - - [05/Nov/2026:10:14:20 +0530] "GET /missing.html HTTP/1.1" 404 191 "python-requests/2.31"
192.168.1.10 - - [05/Nov/2026:10:14:46 +0530] "POST /api/login HTTP/1.1" 401 257 "curl/8.4.0"
10.0.0.5 - - [05/Nov/2026:10:15:25 +0530] "GET /old-page HTTP/1.1" 301 271 "curl/8.4.0"
10.0.0.8 - - [05/Nov/2026:10:18:56 +0530] "GET /favicon.ico HTTP/1.1" 404 199 "Googlebot/2.1"
192.168.1.22 - - [05/Nov/2026:10:20:49 +0530] "GET /old-page HTTP/1.1" 301 206 "Mozilla/5.0"
192.168.1.10 - - [05/Nov/2026:10:24:29 +0530] "POST /api/login HTTP/1.1" 401 234 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:24:52 +0530] "GET /admin HTTP/1.1" 403 266 "Mozilla/5.0"
172.16.0.3 - - [05/Nov/2026:10:25:57 +0530] "GET /favicon.ico HTTP/1.1" 404 284 "curl/8.4.0"
10.0.0.5 - - [05/Nov/2026:10:26:40 +0530] "POST /api/login HTTP/1.1" 401 258 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:30:08 +0530] "GET /Admin/settings HTTP/1.1" 403 218 "Mozilla/5.0"
203.0.113.45 - - [05/Nov/2026:10:30:47 +0530] "GET /old-page HTTP/1.1" 301 185 "Mozilla/5.0"
192.168.1.10 - - [05/Nov/2026:10:31:35 +0530] "GET /api/orders HTTP/1.1" 500 463 "Googlebot/2.1"
192.168.1.10 - - [05/Nov/2026:10:32:41 +0530] "POST /api/orders HTTP/1.1" 503 451 "python-requests/2.31"
```

Q5. Find every request to an admin page, whether it is written as "admin" or "Admin" (case-insensitive search).

Command:grep -i 'admin' access.log

Output:

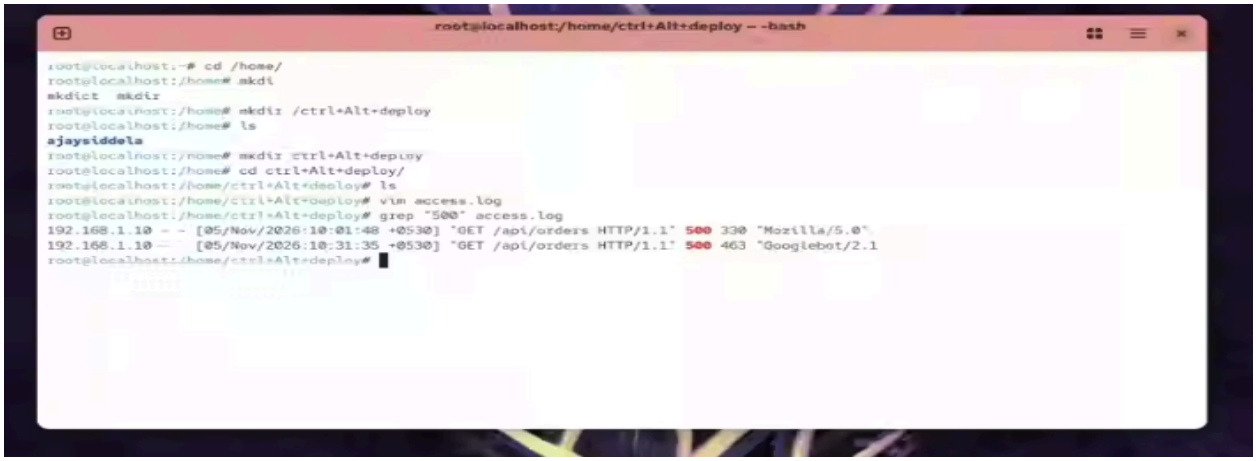
```
ubuntu@ip-172-31-28-203:~$ grep -i 'admin' access.log
10.0.0.8 - - [05/Nov/2026:10:07:37 +0530] "GET /Admin/settings HTTP/1.1" 403 327 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:24:52 +0530] "GET /admin HTTP/1.1" 403 266 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:30:08 +0530] "GET /Admin/settings HTTP/1.1" 403 218 "Mozilla/5.0"
203.0.113.45 - - [05/Nov/2026:10:40:53 +0530] "GET /admin HTTP/1.1" 403 250 "Mozilla/5.0"
192.168.1.15 - - [05/Nov/2026:10:45:10 +0530] "GET /admin HTTP/1.1" 403 349 "Googlebot/2.1"
```

[AJAY SIDDELA]

Q6. Show the line numbers of all requests that returned status 500.

Command:grep “500” access.log

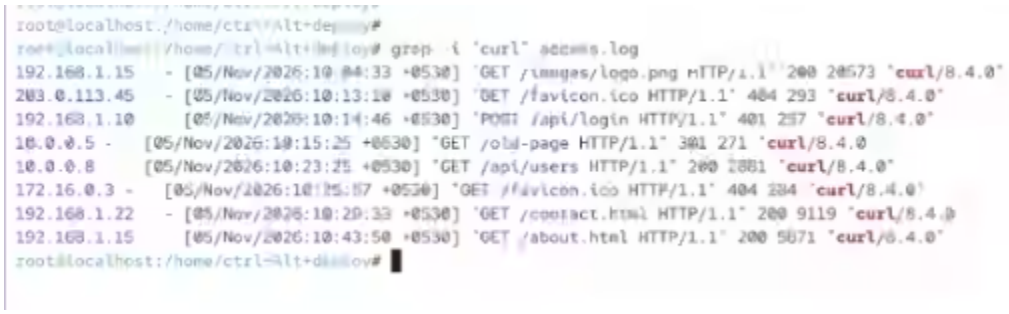
Output:



Q7. Show all requests made using the curl user agent.

Command: grep i “curl” access.log

Output:



Q8. Show all requests to API endpoints (paths starting with /api/).

Command:

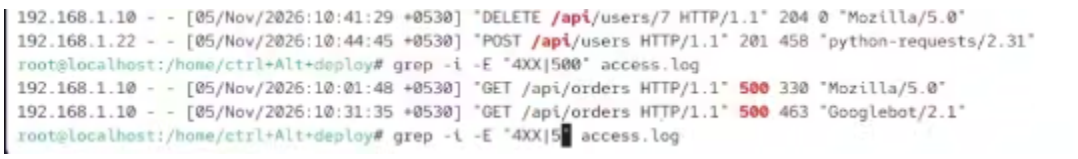
Output:



Q9. Count all error responses, meaning any 4xx or 5xx status code. Hint: use an extended regular expression.

Command: grep -i -E “4xx|500” access.log

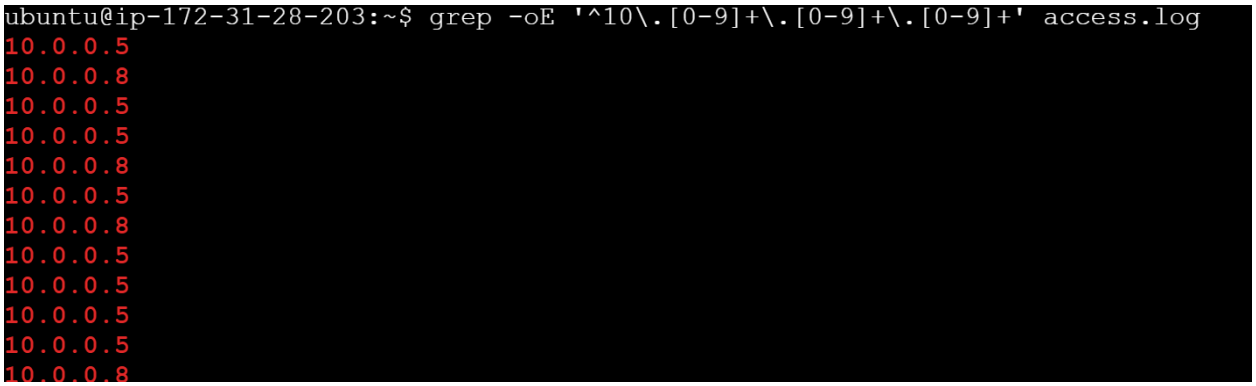
Output:



Q10. Print only the IP addresses that start with 10. (print just the matching part, not the whole line).

Command:

Output:



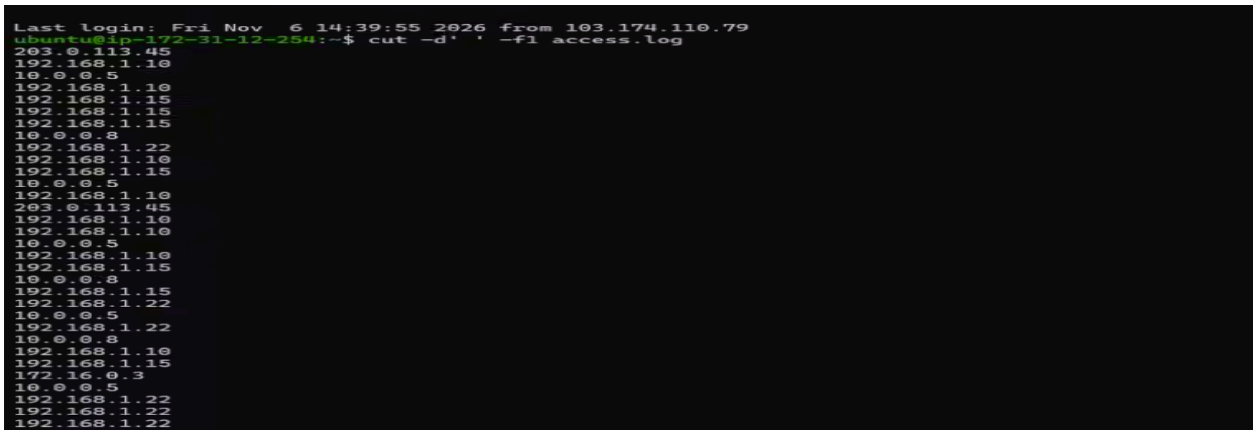
Section B: cut (Questions 11 to 20)

[MAHEEJA]

Q11. Print only the IP address from every line.

Command:

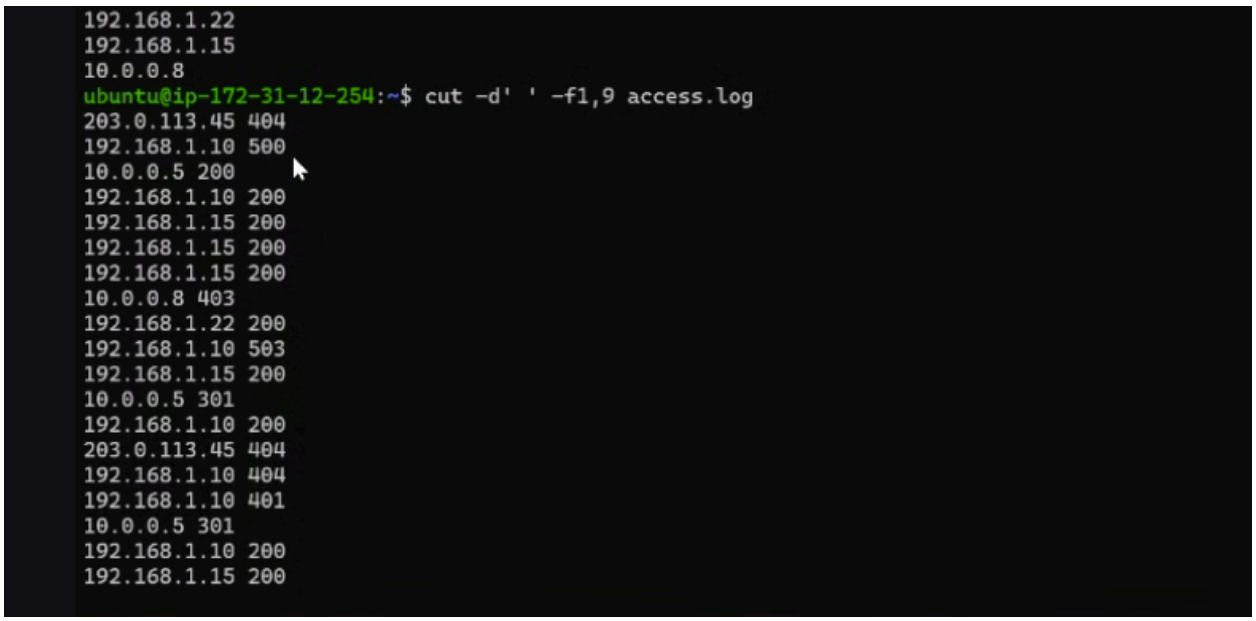
Output:



Q12. Print the IP address and the status code of every request.

Command:

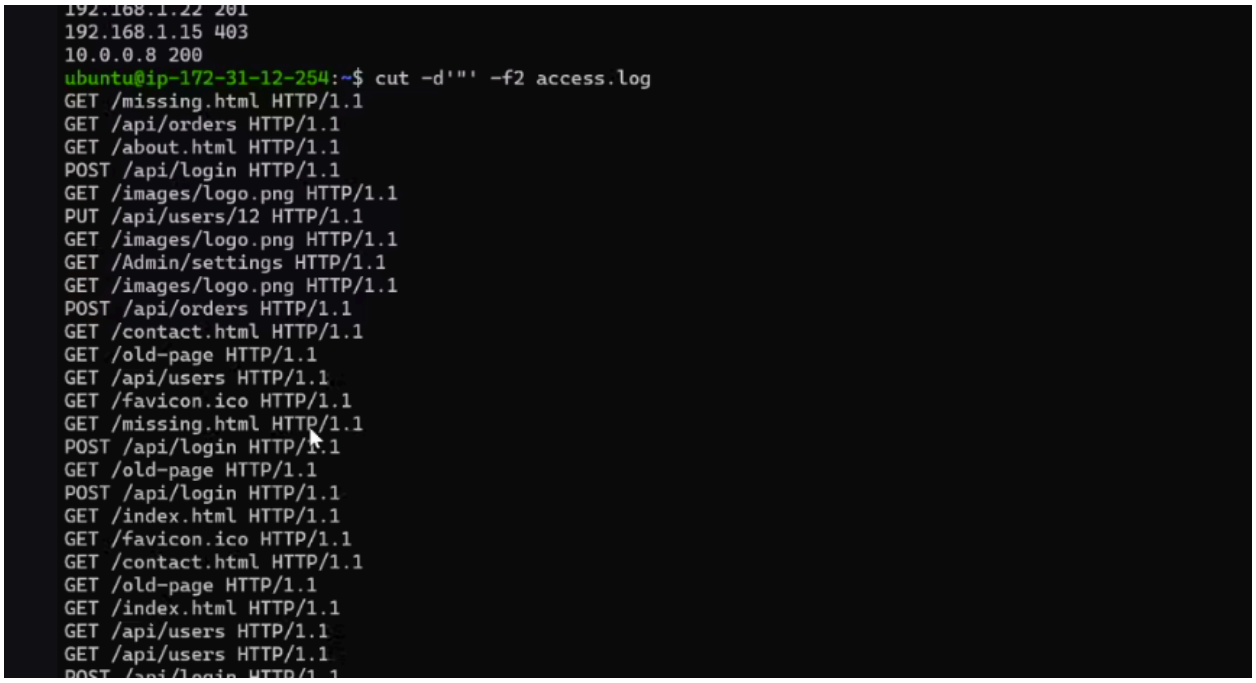
Output:



Q13. Print only the request part that sits inside the first pair of double quotes, for example `GET /index.html HTTP/1.1`.

Command:

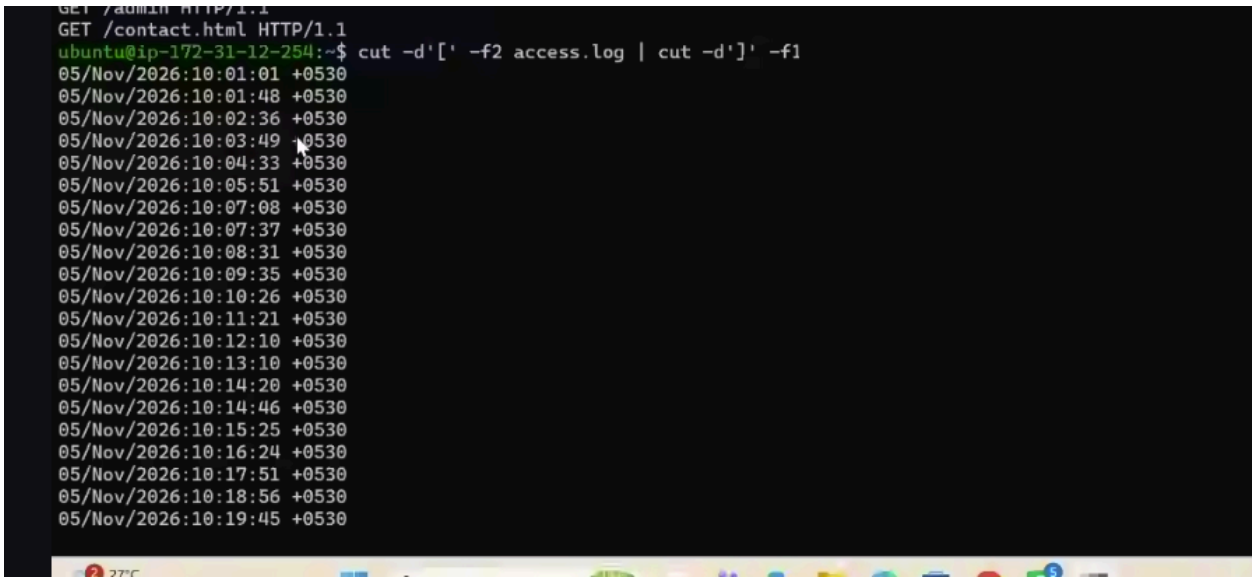
Output:



Q14. Print the full timestamp without the square brackets, for example `05/Nov/2026:10:01:01 +0530`.

Command:

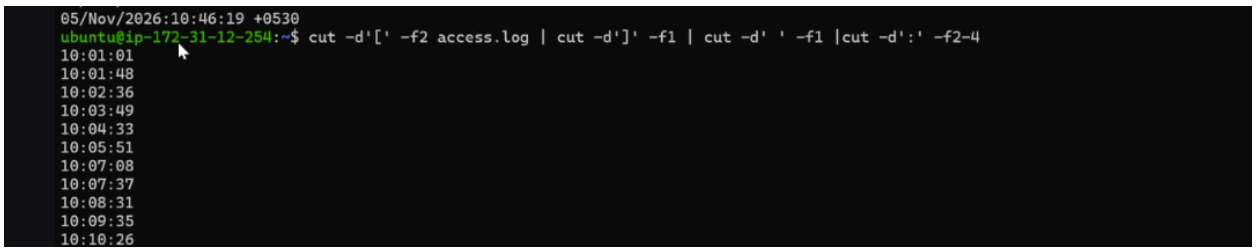
Output :



Q15. Print only the time of each request in HH:MM:SS format, for example `10:01:01`.

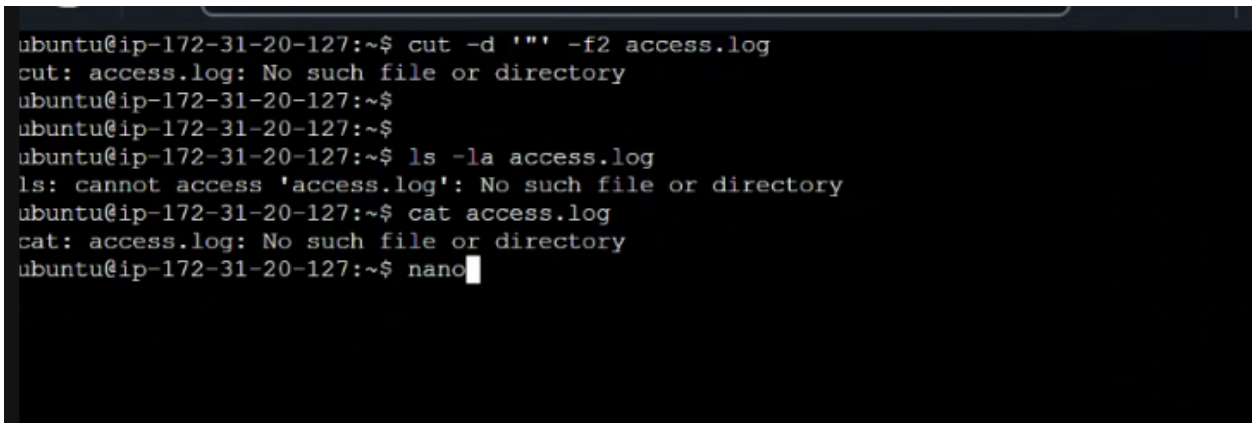
Command:

Output:



[DINESH KONDA]

TROUBLESHOOTING : Didn't paste logs in access.log



Q16. Print only the HTTP method (GET, POST, PUT or DELETE) of each request.

Command:

Output:

```
GET /contact.html HTTP/1.1
ubuntu@ip-172-31-20-127:~$ cut -d '"' -f2 access.log | cut -d ' ' -f1
GET
GET
GET
POST
GET
PUT
GET
GET
GET
POST
GET
GET
GET
GET
GET
POST
GET
```

Q17. Print the first 15 characters of every line.

Command:

Output:

```
ubuntu@ip-172-31-20-127:~$ cut -c1-c15 access.log
cut: range '1-c15' was invalid: failed to parse range
ubuntu@ip-172-31-20-127:~$ cut -c1-15 access.log
203.0.113.45 -
191.168.1.10 -
10.0.0.5 - - [0
192.168.1.10 -
192.168.1.15 -
192.168.1.15 -
192.168.1.15 -
10.0.0.8 - - [0
192.168.1.22 -
192.168.1.10 -
192.168.1.15 -
10.0.0.5 - - [0
192.168.1.10 -
203.0.113.45 -
192.168.1.10 -
192.168.1.10 -
```

Q18. Print the list of unique IP addresses, sorted.

Command:

Output:

```
10.0.0.8 - - [0
ubuntu@ip-172-31-20-127:~$ cut -d ' ' -f1 access.log | sort -u
10.0.0.5
10.0.0.8
172.16.0.3
191.168.1.10
192.168.1.10
192.168.1.15
192.168.1.22
203.0.113.45
ubuntu@ip-172-31-20-127:~$
```

Q19. Count how many requests came from each IP address, with the busiest IP at the top.

Command:

Output:

```
      1 10.0.0.5
ubuntu@ip-172-31-20-127:~$ cut -d ' ' -f1 access.log | sort | uniq -c | sort -nr
     13 192.168.1.10
     11 192.168.1.15
      8 10.0.0.5
      7 192.168.1.22
      4 203.0.113.45
      4 10.0.0.8
      2 172.16.0.3
      1 191.168.1.10
ubuntu@ip-172-31-20-127:~$
```

Q20. Print only the user agent of each request, for example `Mozilla/5.0`.

Command:

Output:

```
ubuntu@ip-172-31-20-127:~$ cut -d '"' -f4 access.log
python-requests/2.31
Mozilla/5.0
Mozilla/5.0
Googlebot/2.1
curl/8.4.0
Mozilla/5.0
Mozilla/5.0
Mozilla/5.0
Googlebot/2.1
Mozilla/5.0
python-requests/2.31
Googlebot/2.1
python-requests/2.31
curl/8.4.0
python-requests/2.31
curl/8.4.0
curl/8.4.0
Mozilla/5.0
Mozilla/5.0
Googlebot/2.1
Googlebot/2.1
Mozilla/5.0
```

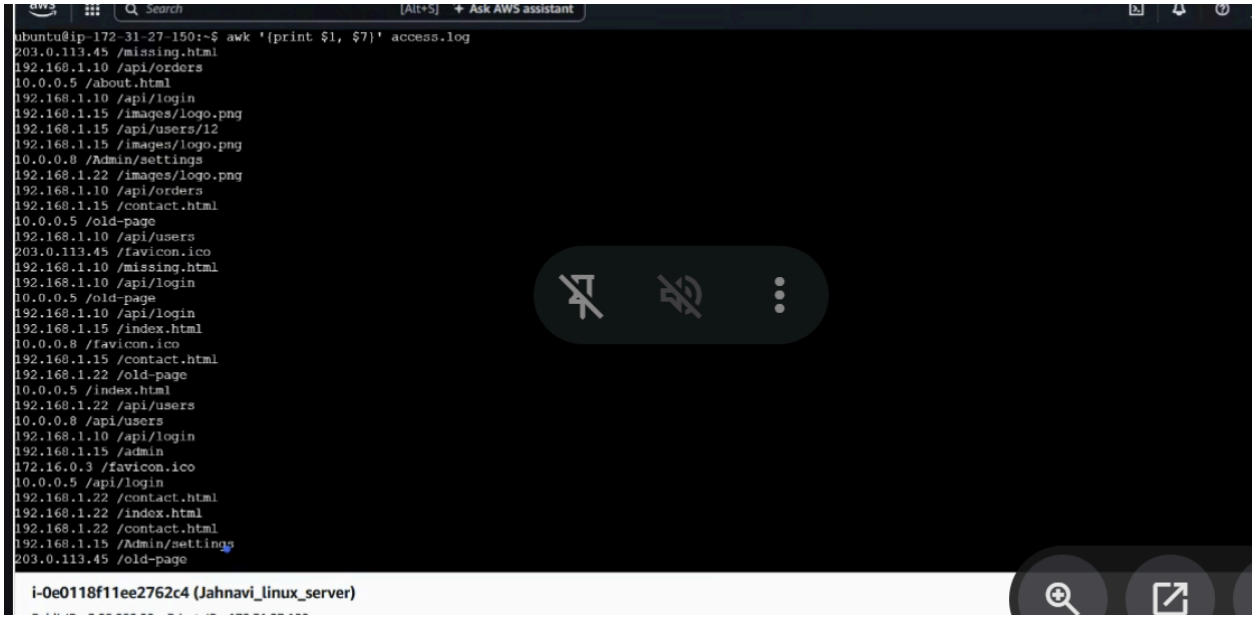
Section C: awk (Questions 21 to 30)

[JAHNAVI]

Q21. Print the IP address and the requested path of every line.

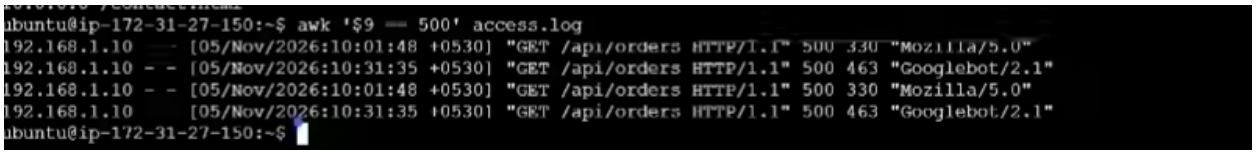
Command:

Output:



Q22. Print only the lines where the status code is exactly 500.

Command:



Q23. Calculate the total number of bytes sent across all requests.

Command:

Output:

```
192.168.1.10 - - [05/Nov/2026:10:31:35 +0530] "GET /api/orders HTTP/1.1" 500 463 "Googlebot/2.1"
ubuntu@ip-172-31-27-150:~$ awk '{sum = sum + $10} END {print sum}' access.log
478719
ubuntu@ip-172-31-27-150:~$
```

Q24. Count how many requests there are for each status code.

Command:

Output:

```
478719
ubuntu@ip-172-31-27-150:~$ awk '{count[$9]++} END {for (code in count) print code, count[code]}' access.log
200 48
201 4
204 4
301 8
401 6
403 10
404 11
500 4
503 4
```

Q25. Print the IP, path and size of every response larger than 10000 bytes.

Command:

Output:

```
ubuntu@ip-172-31-27-150:~$ awk '$10 > 10000 {print $1, $7, $10}' access.log
10.0.0.5 /about.html 10255
192.168.1.10 /api/login 10153
192.168.1.15 /images/logo.png 20573
192.168.1.15 /api/users/12 13815
192.168.1.15 /images/logo.png 21454
192.168.1.22 /images/logo.png 21160
192.168.1.10 /api/login 13082
192.168.1.15 /index.html 13221
192.168.1.22 /contact.html 13649
192.168.1.22 /index.html 13251
10.0.0.5 /about.html 10255
192.168.1.10 /api/login 10153
192.168.1.15 /images/logo.png 20573
192.168.1.15 /api/users/12 13815
192.168.1.15 /images/logo.png 21454
192.168.1.22 /images/logo.png 21160
192.168.1.10 /api/login 13082
192.168.1.15 /index.html 13221
192.168.1.22 /contact.html 13649
192.168.1.22 /index.html 13251
ubuntu@ip-172-31-27-150:~$
```

[PAVAN]

Q26. Calculate the average response size in bytes.

Command:

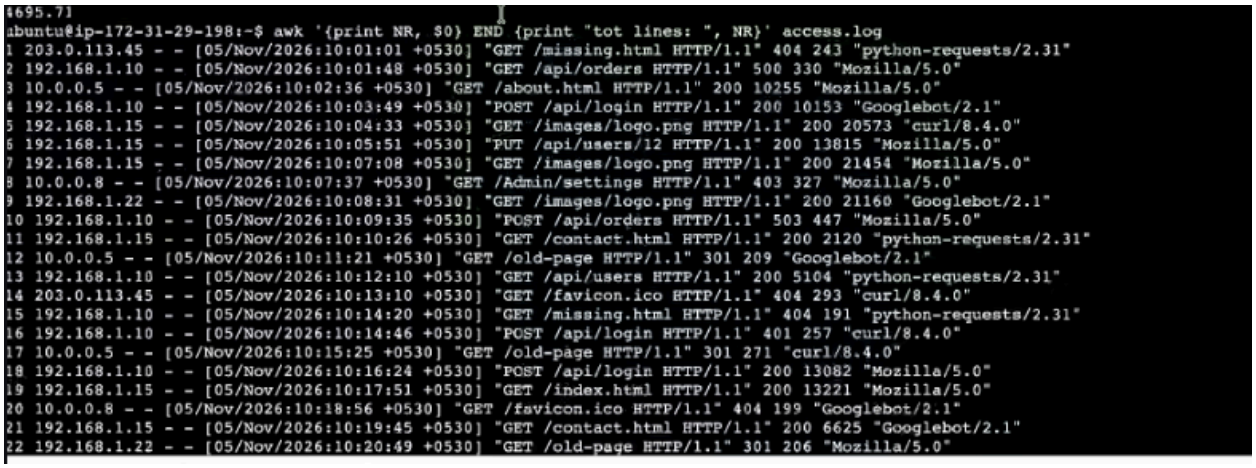
Output:

```
ubuntu@ip-172-31-29-198:~$ awk '{sum += $10} END {if (NR > 0) print sum / NR}' access.log
4695.71
ubuntu@ip-172-31-29-198:~$
```

Q27. Print each line with its line number in front, then print the total number of lines at the end.

Command:

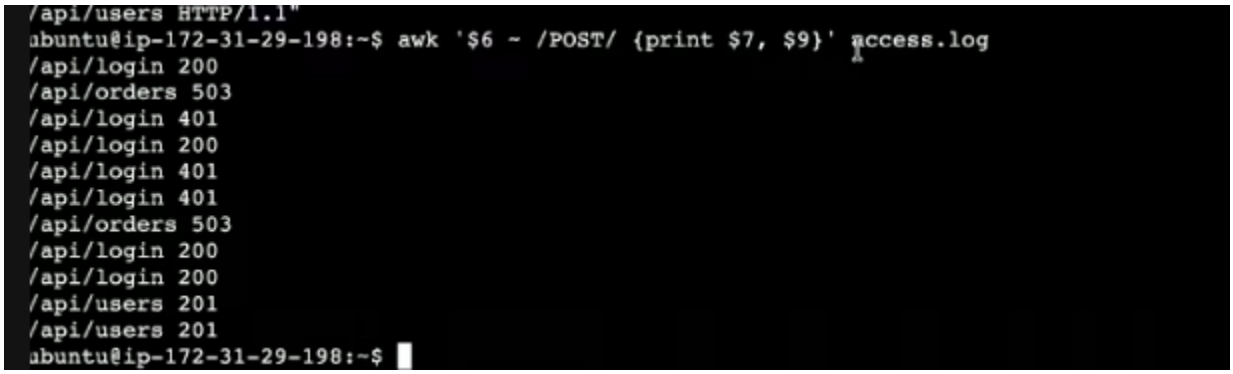
Output:

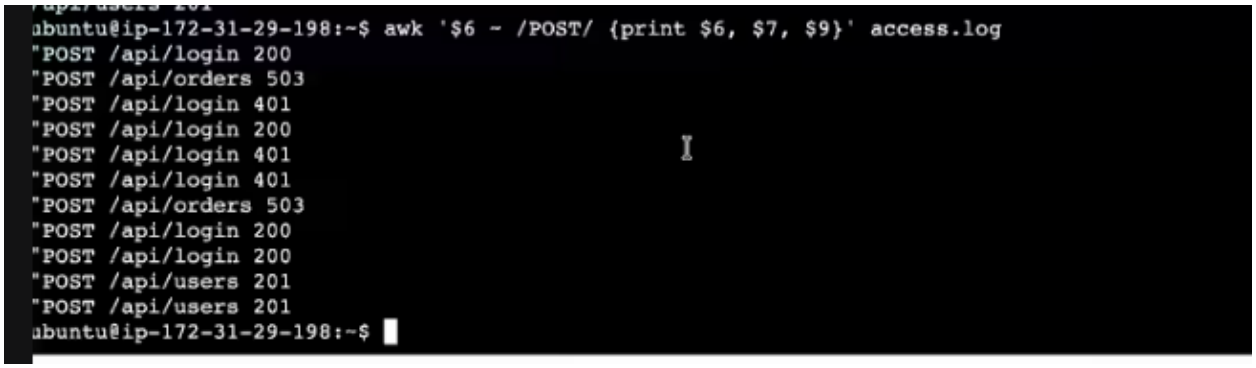


Q28. For POST requests only, print the path and the status code.

Command:

Output:

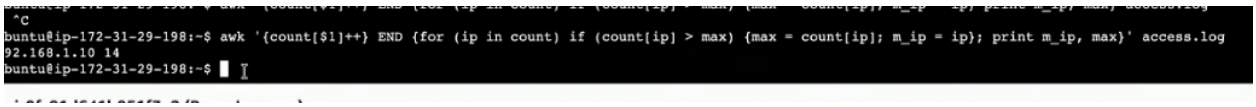




Q29. Find the single IP address that made the most requests, and print it with its count.

Command:

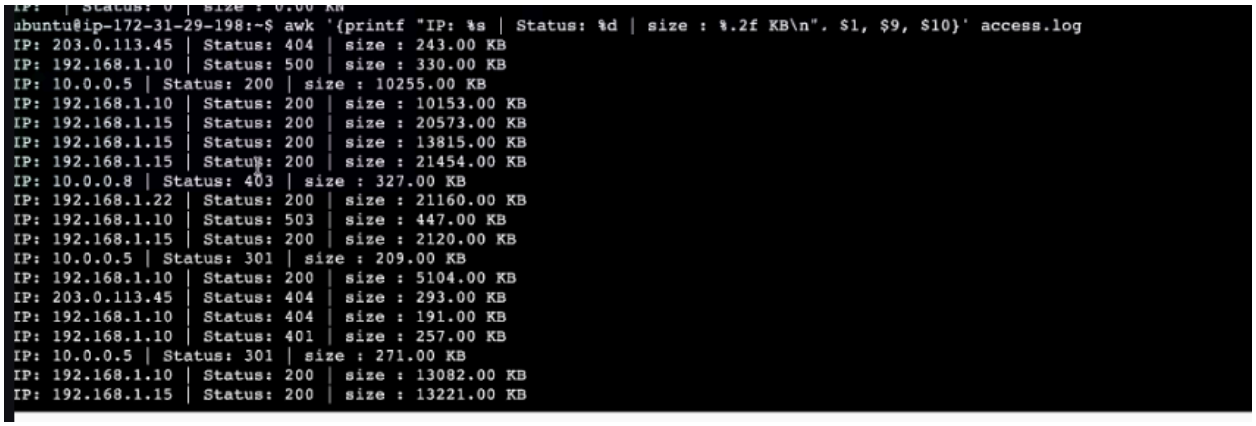
Output:



Q30. Use printf to print a neat report line for every request, in this format: `IP: 10.0.0.5 | Status: 200 | Size: 10.01 KB` (size in KB with 2 decimals).

Command:

Output:



Bonus (optional)

Using one awk command, print the number of failed login attempts (POST /api/login with status 401) for each IP address.

